Lecture 5: Reinforcement learning

Viliam Lisý & Branislav Bošanský

Artificial Intelligence Center
Department of Computer Science, Faculty of Electrical Eng.
Czech Technical University in Prague

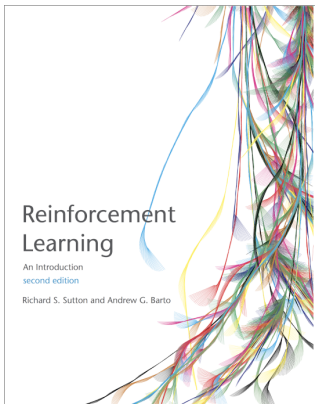
viliam.lisy@fel.cvut.cz

March, 2026

Definition

Wikipedia: Reinforcement learning is "concerned with how intelligent agents ought to take **actions** in an environment in order to maximize the notion of cumulative **reward**"

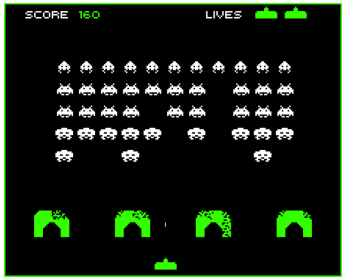
The book: "Reinforcement learning is learning what to do – how to map situations to **actions** – so as to maximize a numerical **reward** signal."

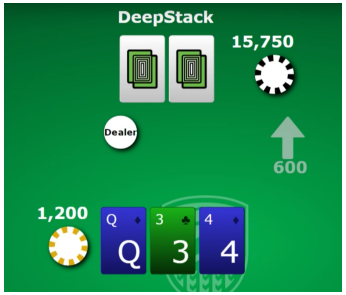


Me: Learning to choose **actions** to optimize **rewards** based on experience – trial and errors.

Motivation – It works

Success stories:









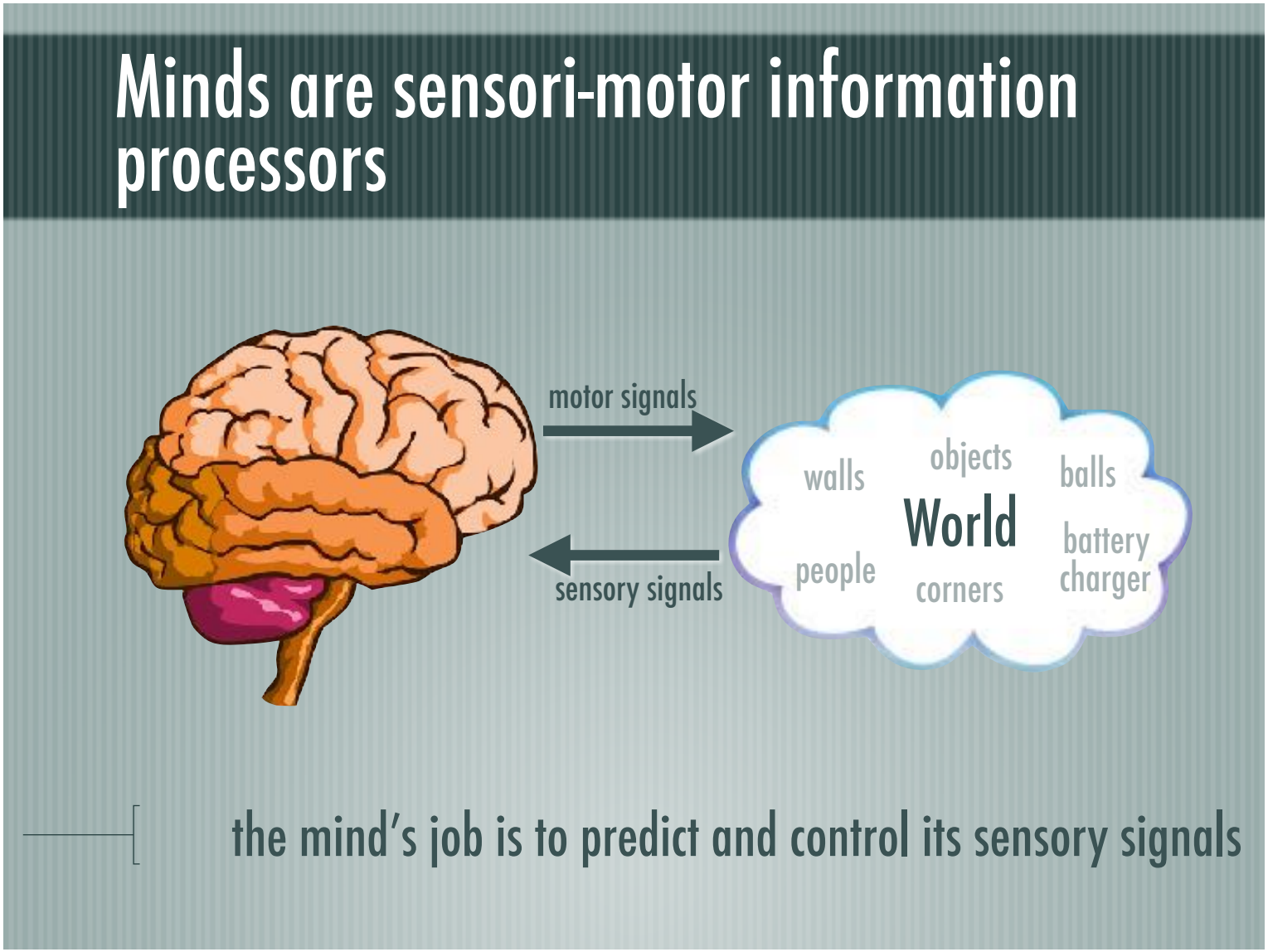






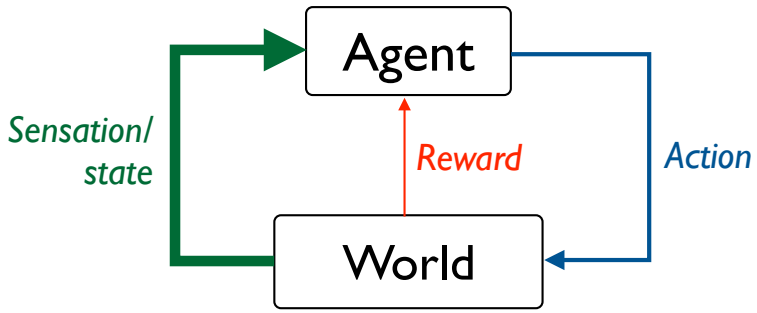
Why is most of this in simulations?

Motivation – It is fundamental



Taken from R. Sutton's slides.

Reinforcement learning is *more autonomous learning*

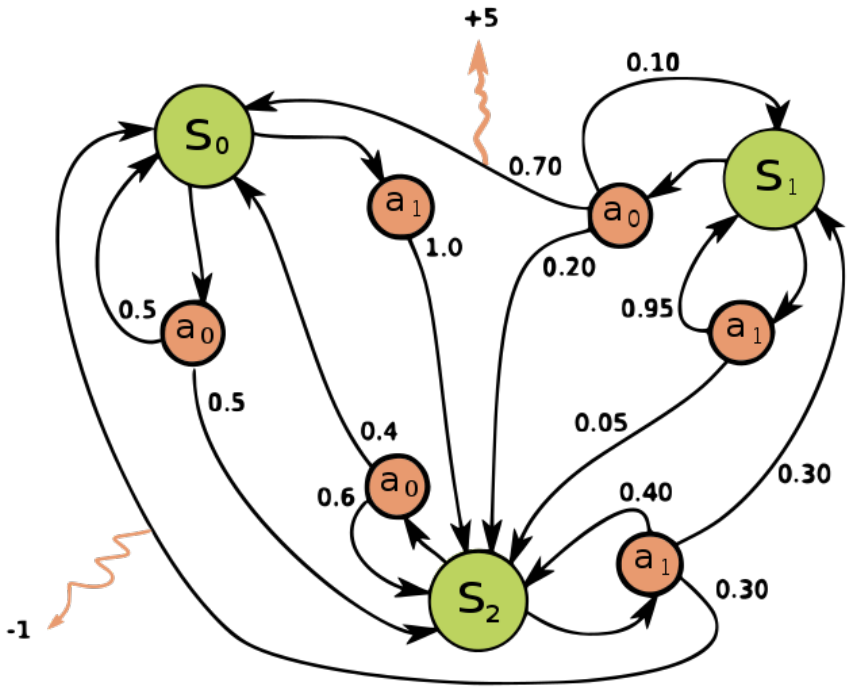


- Learning that requires less input from people
- AI that can learn for itself, during its normal operation

Taken from R. Sutton's slides (and many following are adaptations as well).

Remember MDP

Standard model for Reinforcement Learning problems



Source: Waldoalvarez @ wikimedia

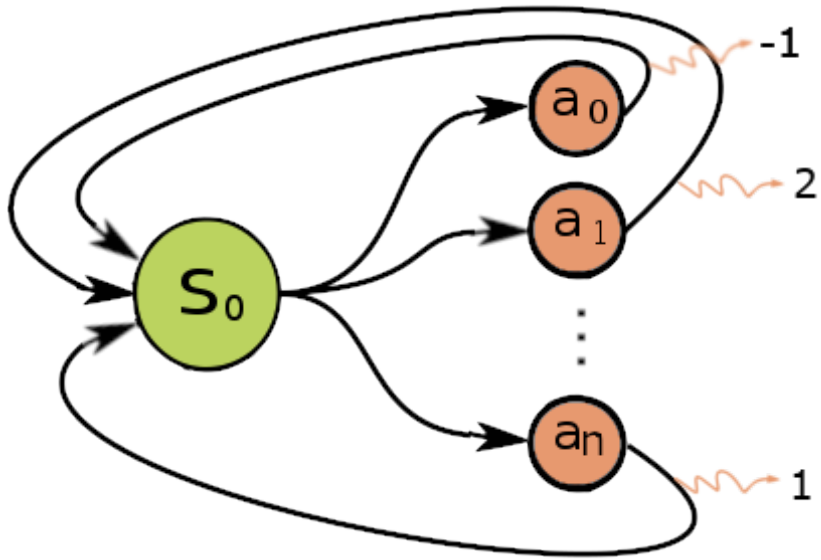
- S – states
- R – rewards
- A – actions
- Discrete steps $t = 0, 1, 2, \ldots$
- Environment *dynamics*

$$p(s', r|s, a) \leftarrow Pr\{S_t = s', R_t = r|S_{t-1} = s, A_{t-1} = a\}$$

- Markov property

Single state MDP: Multi-armed Bandit Problem

All actions $a_1, \ldots, a_n$ lead back to the single state of MDP.



A simple case with many of the RL's fundamental problems.

utility estimation, exploration-exploitation, (non-stationarity)

Why is it called Multi-Armed Bandit Problem



Example problem



Action 1: Reward is always 8

Expected reward: $q_*(1) = 8$

Action 2: 88% chance of 0, 12% chance of 100

Expected reward: $q_*(2) = 12$

Action 3: Uniformly random between -10 and 35

Expected reward: $q_*(3) = 12.5$

Action 4: a third 0, a third 20, and a third from 8-18

Expected reward: $q_*(4) = 13/3 + 20/3 = 11$

Multi-armed Bandit Problem

On each of a sequence of time steps, $t = 1, 2, \ldots, T$ you choose an action A_t from k possibilities, and receive a real-valued reward R_t

The reward depends only on the action taken; it is identically, independently distributed (i.i.d.):

$$q_*(a) \doteq \mathbb{E}[R_t | A_t = a], \forall a \in \{1, \ldots, k\}$$

These true values are **unknown**. The distribution is **unknown**.

Nevertheless, you must maximize your total reward

You must both try actions to learn their values (**explore**), and prefer those that appear best (**exploit**)

The Exploration/Exploitation Dilemma

Suppose you form estimates

$$Q_t(a) \approx q_*(a), \forall a \quad \text{action-value estimates}$$

Define the **greedy action** at time t as

$$A_t^* \doteq \arg\max_a Q_t(a)$$

If $A_t = A_t^*$ then you are *exploiting*
If $A_t \neq A_t^*$ then you are *exploring*

You can't do both, but you need to do both

You can never stop exploring, but maybe you should explore less with time. Or maybe not.

MAB Applications

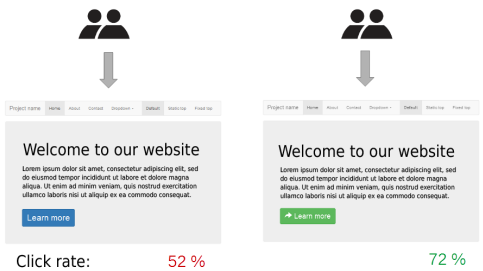


Clinical trials

Actions = treatments

Reward = outcome

Exploration means giving a potentially worse drug – ethical stakes!

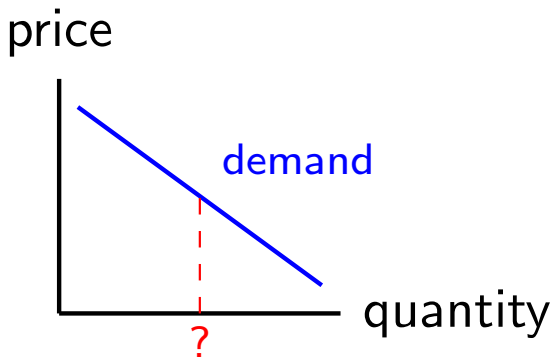


A/B testing

Actions = page variants

Reward = click

Pure A/B test wastes traffic; adaptive allocation saves money.



Dynamic pricing

Actions = price points

Reward = revenue

Demand curve unknown; must explore to find optimum.

Action-Value Methods

Methods that learn action-value estimates and nothing else

For example, estimate action values as sample averages:

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}}$$

The sample average estimates converge to the true values
If the action is taken an infinite number of times

$$\lim_{N_t(a)\to\infty} Q_t(a) = q_*(a)$$

Where $N_t(a)$ is the number of times action a has been taken by time t.

ϵ-Greedy Action Selection

In greedy action selection, you always exploit

In ϵ-greedy, you are usually greedy, but with probability ϵ you instead pick an action at random (possibly the greedy action again)

This is perhaps the simplest way to balance exploration and exploitation

Algorithm ϵ-Greedy:

Initialize, for $a = 1$ to k:

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

Repeat forever:

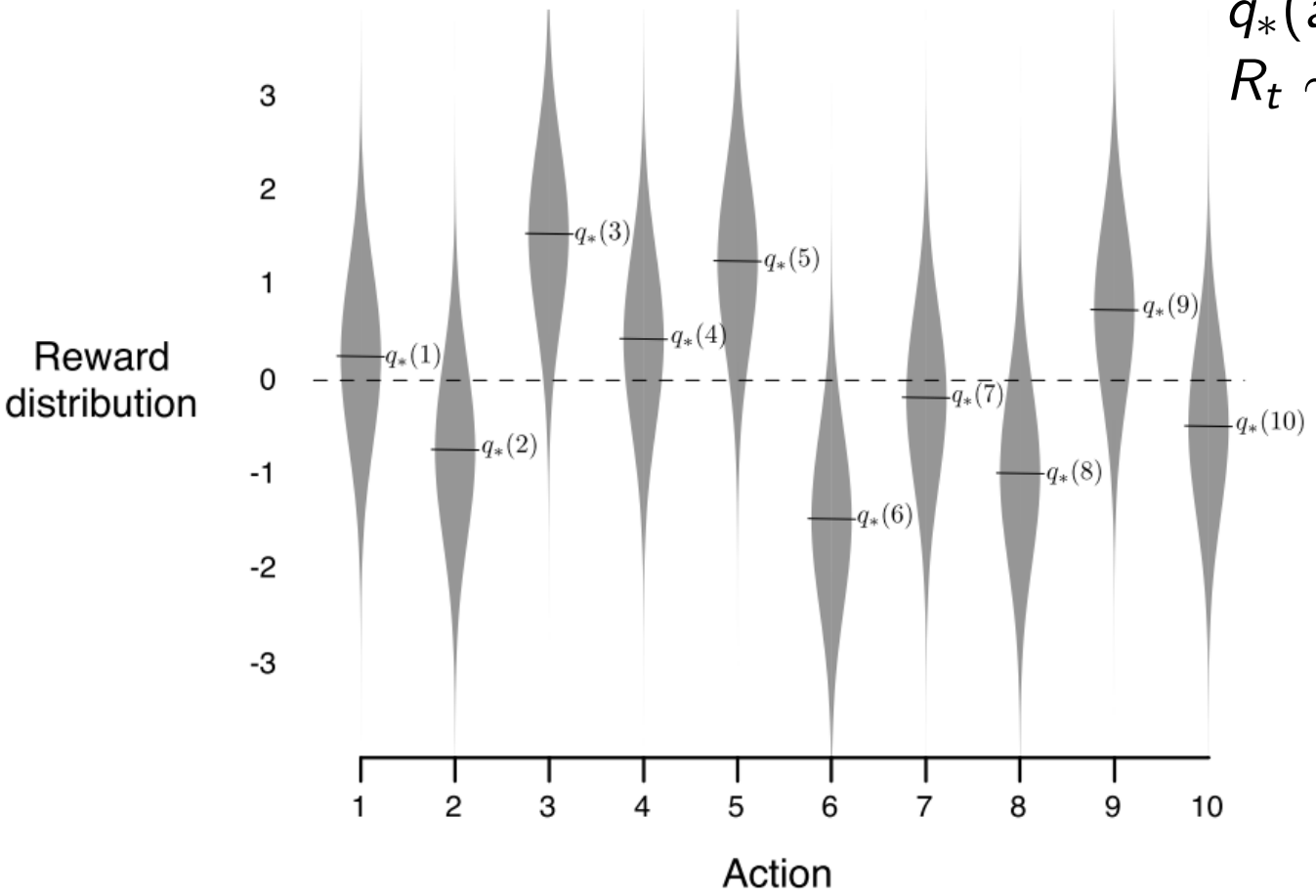
$$A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

$R \leftarrow bandit(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}\left[R - Q(A)\right]$

One Task from the 10-armed Testbed

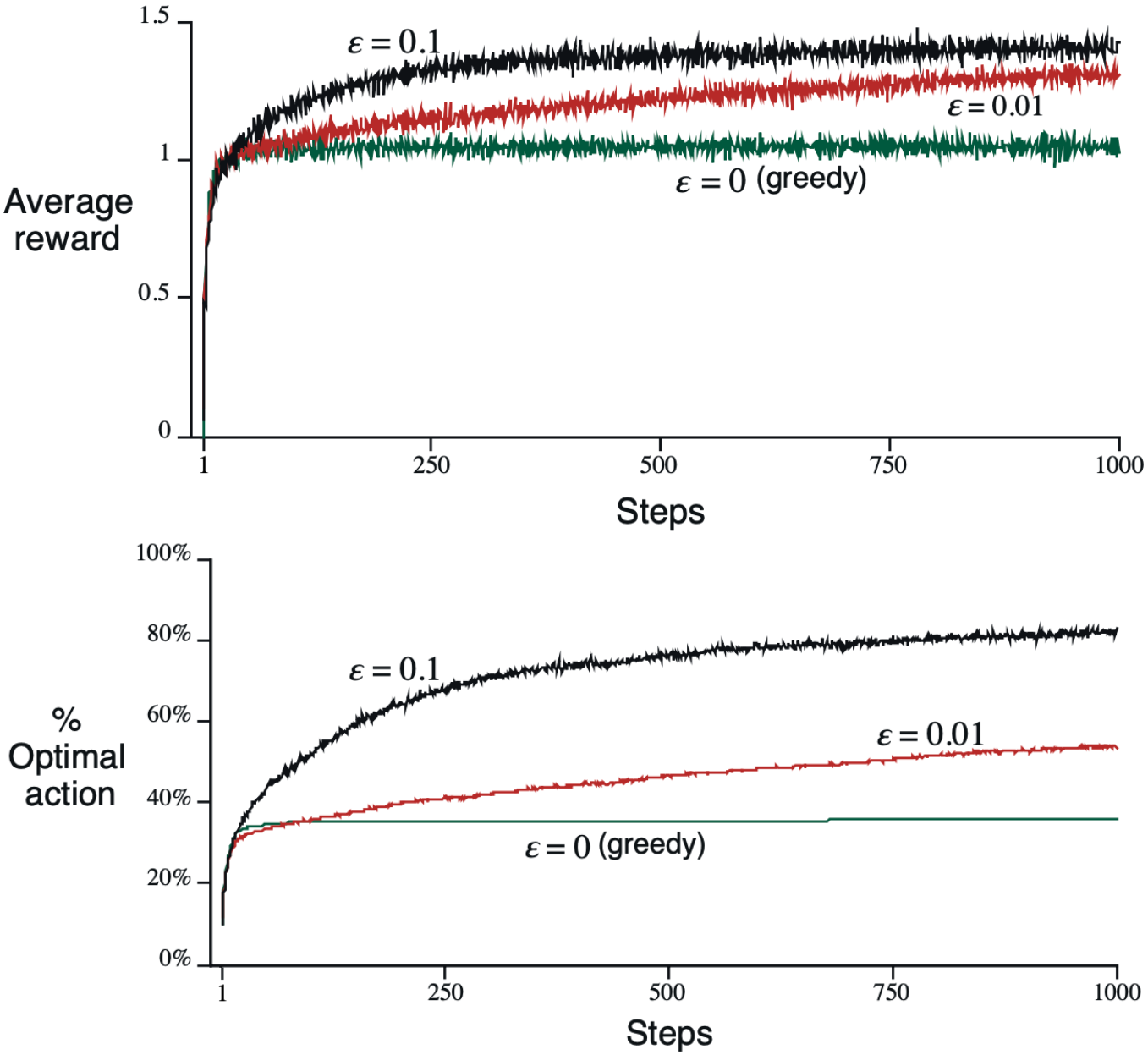


$$q_*(a) \sim \mathcal{N}(0, 1)$$
$$R_t \sim \mathcal{N}(q_*(a), 1)$$

Run for 1000 steps

Repeat the whole thing 2000 times with different bandit tasks

ϵ-Greedy Methods on the 10-Armed Testbed



Averaging $\rightarrow$ Learning Rule

To simplify notation, let us focus on one action

$$Q_n \doteq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1}$$

How can we do this incrementally (without storing all the rewards)?

Could store a running sum and count (and divide), or equivalently:

$$Q_{n+1} = Q_n + \frac{1}{n}\left[R_n - Q_n\right]$$

This is a standard form for learning/update rules:

$$\mathit{NewEstimate} \leftarrow \mathit{OldEstimate} + \mathit{StepSize}\left[\mathit{Target} - \mathit{OldEstimate}\right]$$

Derivation of incremental update

$$Q_n \doteq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1}$$

$$\begin{aligned}
Q_{n+1} &= \frac{1}{n}\sum_{i=1}^{n} R_i \\
&= \frac{1}{n}\left(R_n + \sum_{i=1}^{n-1} R_i\right) \\
&= \frac{1}{n}\left(R_n + (n-1)\frac{1}{n-1}\sum_{i=1}^{n-1} R_i\right) \\
&= \frac{1}{n}\Big(R_n + (n-1)Q_n\Big) \\
&= \frac{1}{n}\Big(R_n + nQ_n - Q_n\Big) \\
&= Q_n + \alpha_n\Big[R_n - Q_n\Big],
\end{aligned}$$

Standard stochastic approximation convergence conditions

To assure convergence with probability 1:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \qquad \text{and} \qquad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

e.g., $\alpha_n \doteq \frac{1}{n}$

not $\alpha_n \doteq \frac{1}{n^2}$

if $\alpha_n \doteq n^{-p}, p \in (0.5, 1]$ then convergence is at the optimal rate $O(1/\sqrt{n})$

However, often used with a different schedule, even $\alpha_n = c$.

Optimistic Initial Values

Algorithm ϵ-Greedy:

Initialize, for $a = 1$ to k:

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

Repeat forever:

$$A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

$R \leftarrow bandit(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}\left[R - Q(A)\right]$

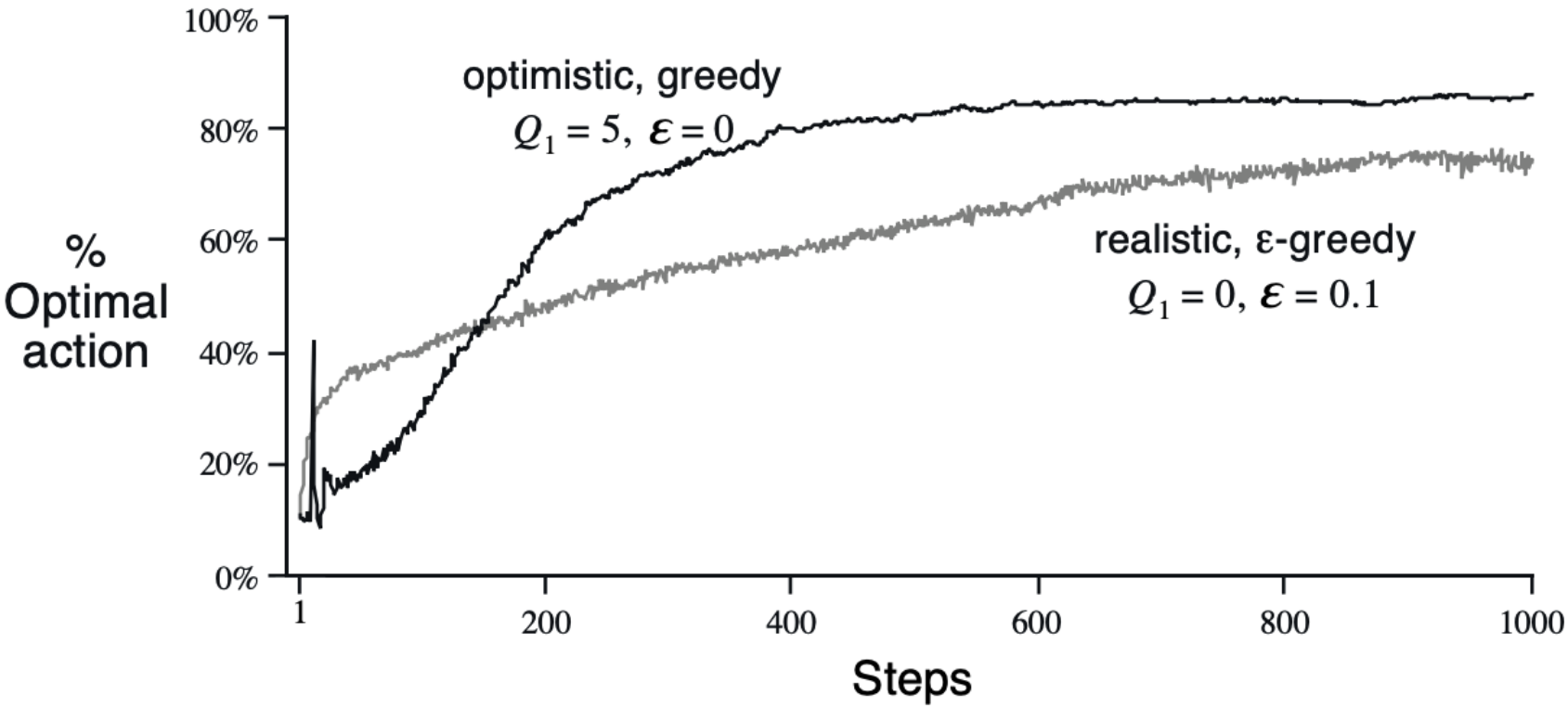
What happens with exploration when we set $Q_1 \neq 0$?

$$Q'_{n+1} = \frac{Q_1 + R_1 + R_2 + \cdots + R_n}{n} = \frac{Q_1}{n} + Q_{n+1}$$

Optimistic Initial Values

The estimates depend on $Q_1(a)$, i.e., they are biased.

Suppose we initialize the action values **optimistically** ($Q_1(a) = 5$),



Is it guaranteed to eventually play the optimal action?

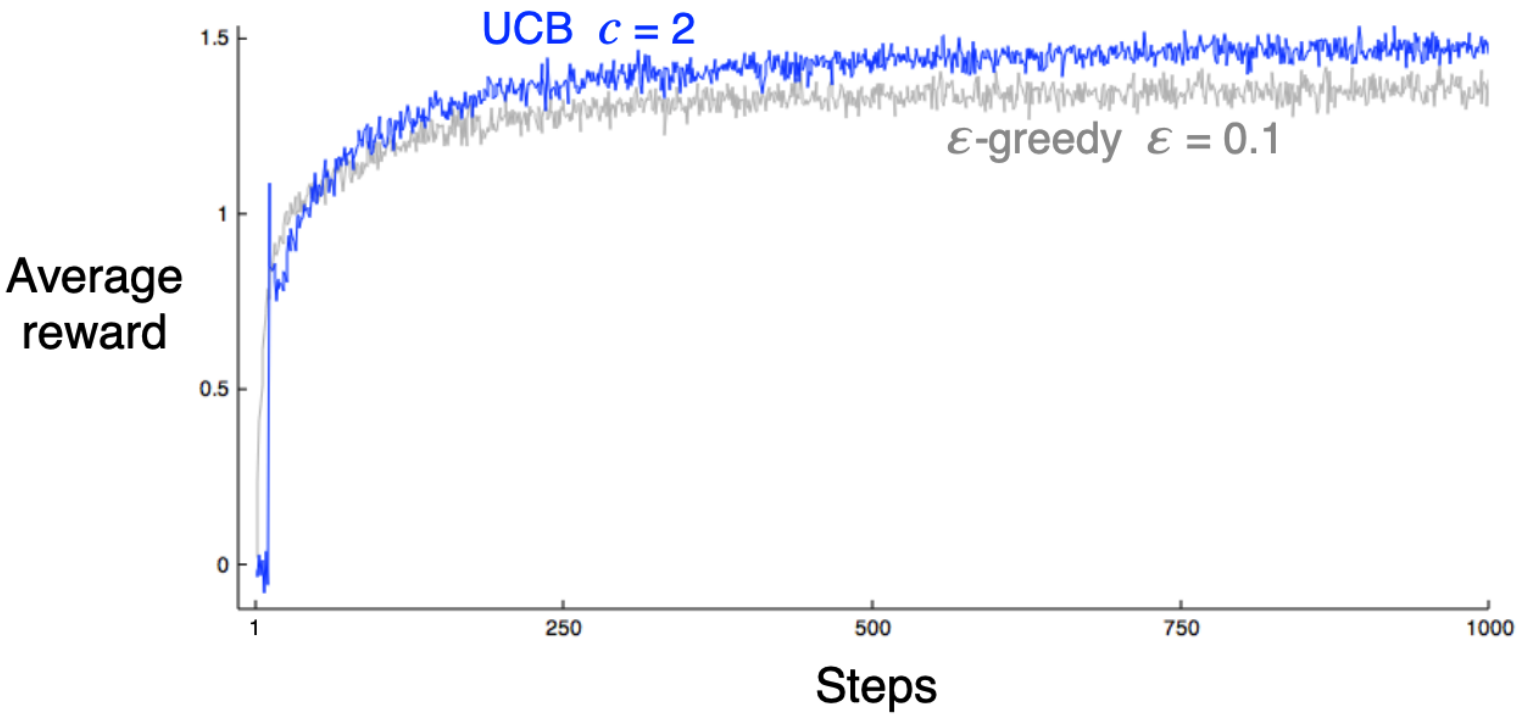
Upper Confidence Bound (UCB) action selection

A clever way of reducing exploration over time

Estimate an upper bound on the true action values

Select the action with the largest (estimated) upper bound

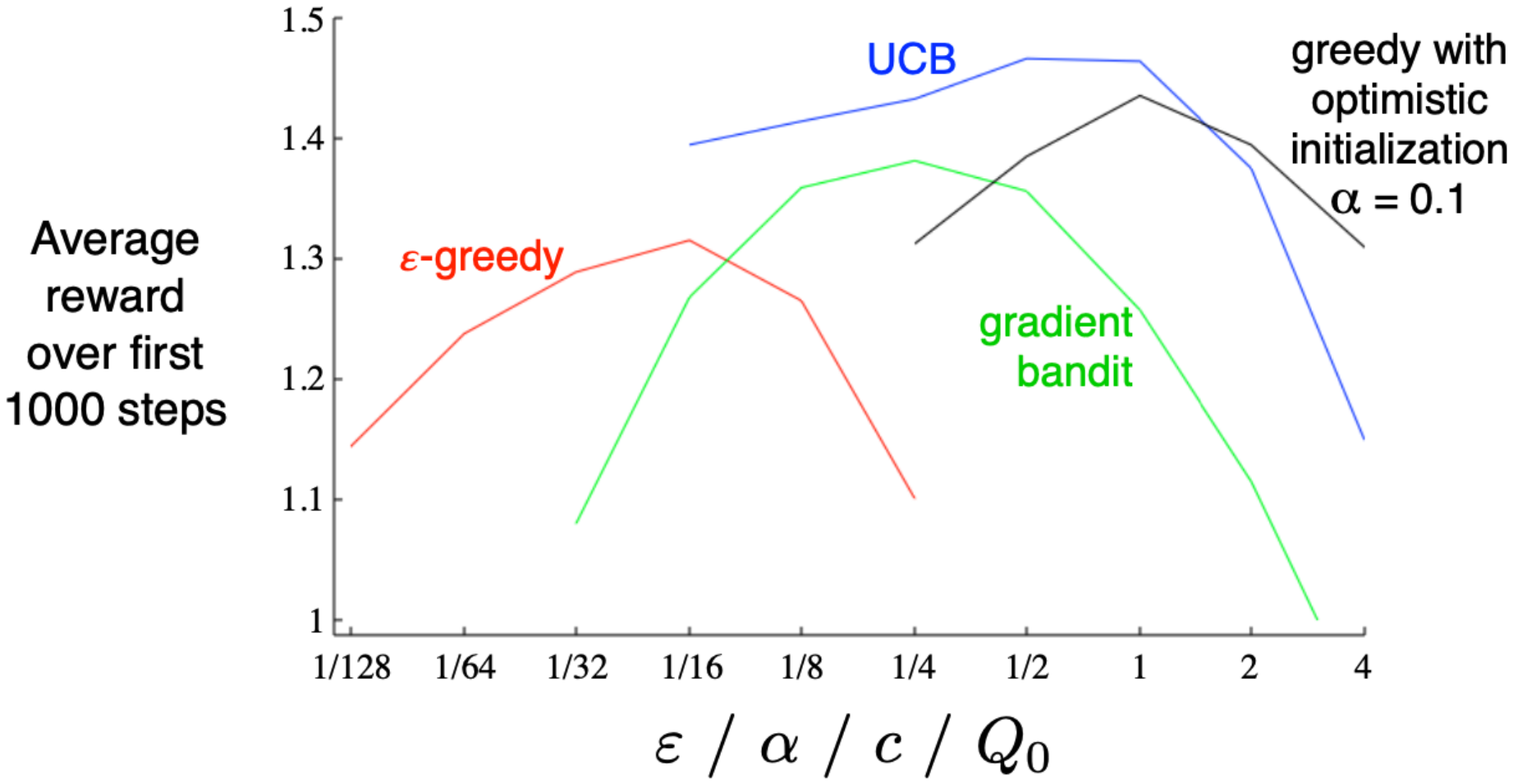
$$A_t \doteq \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\log t}{N_t(a)}} \right]$$



Demo

Live demo in the browser.

Comparison of Bandit Algorithms



Showing advertisments

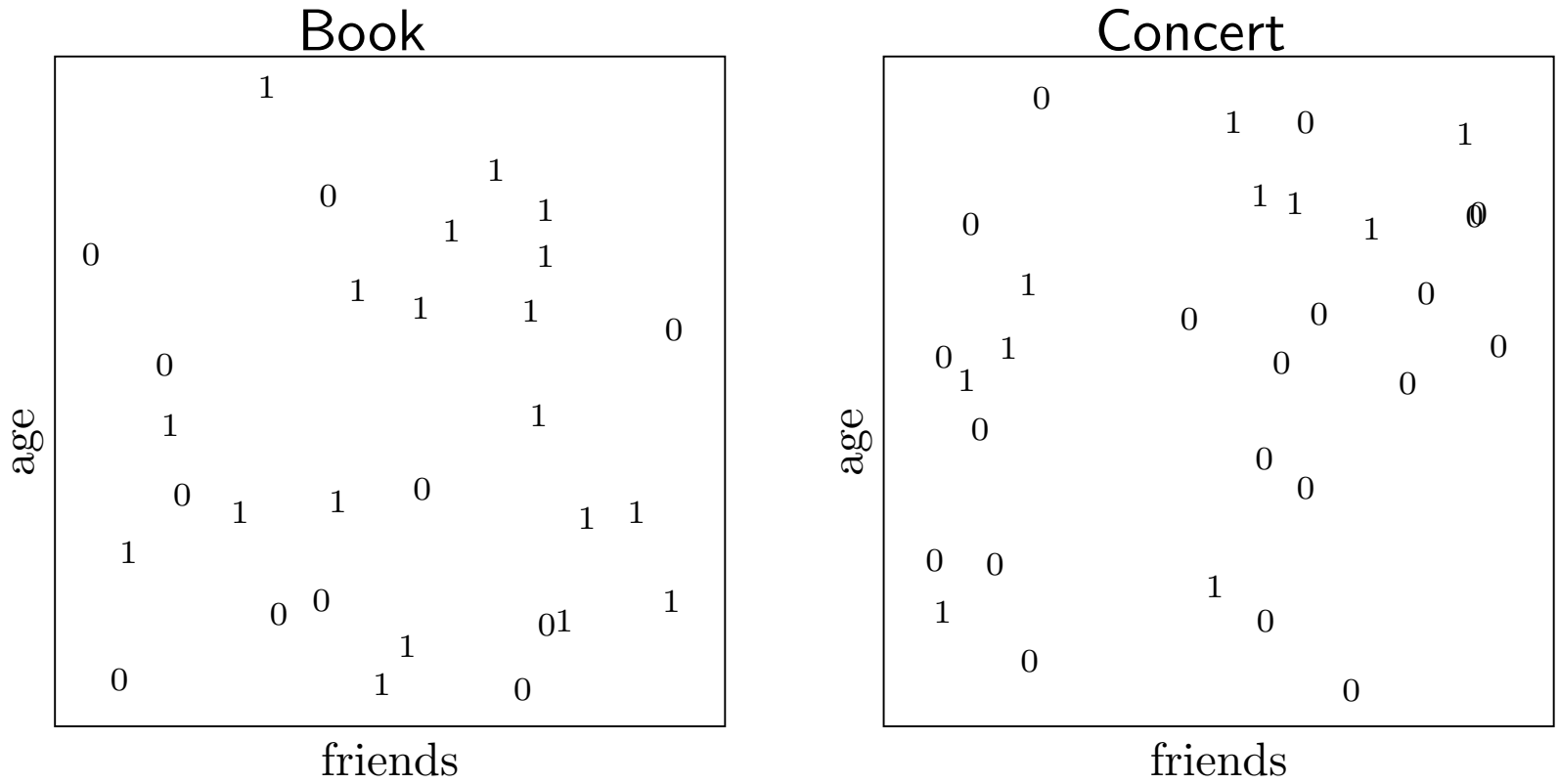


Ads to show: Concert, Book, Movie, Vacation, etc.

Advertisers collect: Age, #friends, gender, etc.
Can we use that?

Contextual Bandit Problem

Assumption: Similar contexts have similar preferences.



$$\arg\max_a \mu_t(\vec{c}, a) + C\sqrt{\frac{\log t}{n_t(\vec{c}, a)}}$$

Contextual Bandits with Function Approximation

Problem: with continuous or high-dimensional context $\vec{c}$, we cannot maintain counts $n_t(\vec{c}, a)$ or averages $\mu_t(\vec{c}, a)$ for each context.

Solution: use a function approximator $\hat{q}(\vec{c}, a; \theta)$ to generalize across contexts.

- Each round we observe context $\vec{c}$, choose action a, receive reward r
- This gives us a training example: input $(\vec{c}, a)$, target r
- We minimize $(\hat{q}(\vec{c}, a; \theta) - r)^2$ – standard regression!

The function approximator can be any ML model (linear, decision tree, neural network, ...)

Exploration with Function Approximation

The UCB bonus $\sqrt{\log t / n_t(\vec{c}, a)}$ relied on visit counts – these are not available with function approximation over continuous contexts.

Practical alternatives:

- ϵ-greedy on the learned values $\hat{q}(\vec{c}, a; \theta)$
- Train an ensemble of models, explore where they disagree

This is a general tension in RL:
function approximation helps generalization but makes principled exploration harder

Bandits Summary

These are all simple methods

- but they are sufficient – we will build on them
- we should understand them completely
  - there is a lot of theory, e.g., upper/lower bounds
- there are still open questions

Our first algorithms that learn from evaluative feedback

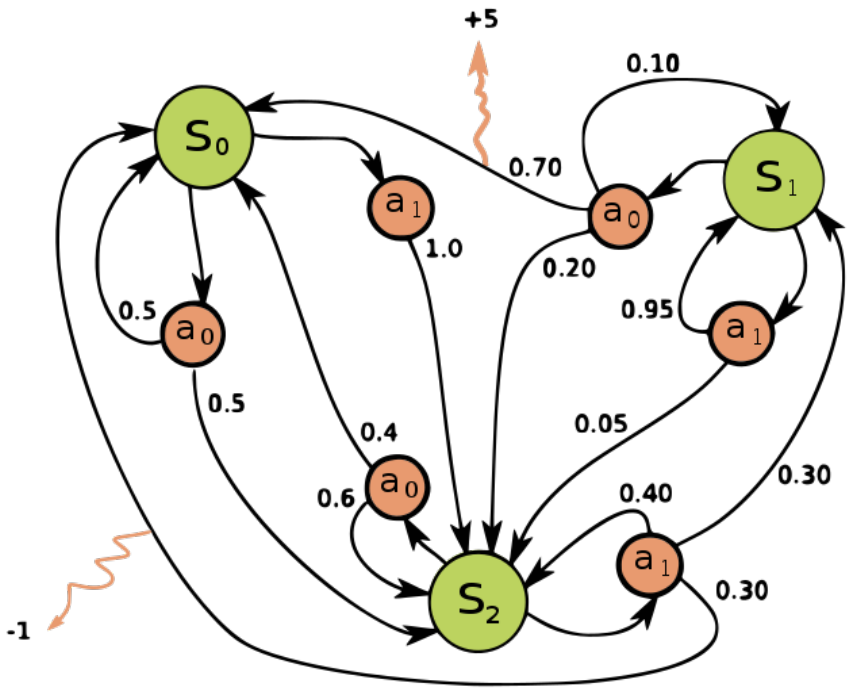
- and thus must balance exploration and exploitation
- that learn to maximize reward by trial and error

Contextual bandits add ML and are heavily used



Back to MDPs

Standard model for Reinforcement Learning problems



Source: Waldoalvarez @ wikimedia

- S – states
- R – rewards
- A – actions
- Discrete steps $t = 0, 1, 2, \ldots$
- Environment *dynamics*

$$p(s', r|s, a) \leftarrow Pr\{S_t = s', R_t = r|S_{t-1} = s, A_{t-1} = a\}$$

Running Example: Warehouse Robot



- **States:** (location, carrying?) – e.g., (Loading, has package)
- **Actions:** turn left/right, drive 1m, pick up, put down, charge
- **Rewards:** $+10$ deliver package, -1 per step, -100 if battery dies

The Agent Learns a Policy

Policy at step t, denoted π_t, maps from states to actions.

$$\pi_t(a|s) = \text{ probability that } A_t = a \text{ when } S_t = s$$

Special case are **deterministic** policies.

$$\pi_t(s) = \text{ the action taken with } prob = 1 \text{ when } S_t = s$$

- Reinforcement learning methods specify how the agent changes its policy as a result of experience
- Roughly, the agent's goal is to get as much reward as it can **over the long run**.

Warehouse robot in state (Aisle, low battery, has package):

Deterministic: $\pi(s) = \text{charge}$ (safe but not informative)

Stochastic: $\pi(\text{charge}|s) = 0.8$, $\pi(\text{drive 1m}|s) = 0.2$
(sometimes risk it to learn low-battery consumption)

Return

Suppose the sequence of rewards after step t is:

$$R_{t+1}, R_{t+2}, R_{t+3}, \ldots$$

What do we maximize?

At least three cases, but in all of them, we seek to maximize the **expected return**, $\mathbb{E}\, G_t$, on each step t.

- **Total reward**, G_t = sum of all future reward in the episode
- **Discounted reward**, G_t = sum of all future *discounted* reward
- **Average reward**, G_t = average reward per time step

Episodic Tasks

Episodic tasks: interaction breaks naturally into episodes, e.g., plays of a game, trips through a maze, **delivering one package**

In episodic tasks, we almost always use simple total reward:

$$G_t = R_{t+1} + R_{t+2} + \cdots + R_T,$$

where T is a final time step at which a **terminal state** is reached, ending an episode.

Continuing Tasks

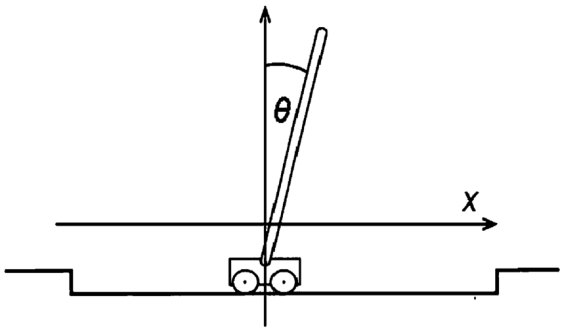
Continuing tasks: interaction does not have natural episodes, but just goes on and on... e.g., **warehouse robot operating nonstop**

In this class, for continuing tasks we will always use *discounted return*:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

where $0 \leq \gamma \leq 1$, is the **discount rate**.
shortsighted $0 \leftarrow \gamma \rightarrow 1$ farsighted
Typically, $\gamma = 0.9$

An Example: Pole Balancing



(image from Ma&Likharev 2007)

Avoid **failure**: the pole falling beyond a critical angle or the cart hitting end of track

As an **episodic task** where episode ends upon failure:

reward $= +1$ for each step before failure

$\Rightarrow$ return $=$ number of steps before failure

As a **continuing task** with discounted return:

reward $=$ -1 upon failure; 0 otherwise

$\Rightarrow$ return $= -\gamma^k$, for k steps before failure

In either case, return is maximized by avoiding failure for as long as possible.

Summary

RL is a set of methods to learn a policy from an interaction with environment

The goal is to maximise return derived from immediate rewards

The simplest RL problem is the multi-armed bandit problem

- exploration vs. exploitation problem
- ϵ-greedy, optimistic initialisation, UCB

Canonical model of RL problems is MDP